Can’t Spill the Beans

Task : Build a reliable bonus point and discount system

Background

Around the corner of the quiet neighborhood street, sits the cozy little coffee shop. Every morning, the smell of freshly ground beans drifts through the air as regular customers line up for their favorite cups to kick-start their day.

To appreciate its regular customers, the shop introduces a new **membership program**. Customers can register as members, and every time they buy coffee, they earn bonus points based on how much they spend. For every **50 taka**, they earn **1 bonus point**.

These points accumulate and can be redeemed for discounts. **1 point gives 1 taka off** the bill.

At first, the staff manually ran the program. A notebook behind the counter, rough calculations, and handwritten totals.

One busy afternoon, the shop gets crowded. Orders pile up. A staff member forgets to add bonus points for a customer. Another customer redeems more points than they actually have. The coffee menu price was updated on paper but not communicated to everyone. By closing time, the bonus records no longer matched reality.

The owner realizes that even his small coffee shop, being so busy that it is, needs a reliable digital system that can track coffee items, members, bonus points and discounts accurately.

The Challenge

The owner of the coffee shop wants a system that -

> **reliably accumulates bonus points and calculates discounts.**

The system will:

- Store and manage related data (coffee items, members, their bonus points).
- Expose APIs for daily operations.
- **Calculate bonus points** upon purchase.
- **Calculate discounts and discounted prices accurately** when bonus points are redeemed.

So the owner of the coffee shop hires your team to build a reliable system for his popular little coffee place, so that he can run the membership program smoothly to keep his customers happy.

How well you can help him to build this system reliably, will be considered as your performance for this mock preliminary round.

To help you get started, we have designed a list of APIs that you need to develop, and defined a list of tasks you need to complete to do well under our performance metrics.

You can access the required APIs in the following link -

[Required APIs for Bonus Point System]

And the tasks defined are listed below.

Task: Design the Database

Design a minimal, normalized relational database based on the given API requirements and the system goals.

Your design should -

- Support all the required APIs.
- Avoid redundancy or unnecessary complexity.
- Follow good relational design principles (normalization, clear relationships, appropriate constraints).

For this task, you must submit -

- An **md** file with Schema Documentation
    - Clearly describe all database tables.
    - Include fields, data types, primary keys, foreign keys, and relationships.

Task: Develop the Server

In this task -

You need to develop a fully working server that implements the required APIs, with a solid logical flow underneath, capable of handling staff requests to accurately maintain system features.

Keep the following key points in mind while building your core component -

Docker Containerization:

You will be required to run your server and database fully with docker. This is **mandatory** to evaluate your server functionality.
Upon running the server, your database will contain empty tables only.
We will populate the tables with API calls and then test how they function.
(more on this later)

API Coverage: Design a server that will serve all required APIs.

Choice of Framework: You are free to choose a framework of your own (spring, node js, django etc).

PORT Requirement :

For functional testing,

You must run the server on port 8000 in localhost

After running the server, available working APIs should be like this -

- POST localhost:8000/coffees
- GET localhost:8000/coffees

  and so on.

TEST Requirements: For simplicity of functional testing of your server -

- **Do not use any unnecessary secrets** in your code
- Do not use any env files or any other secret configuration files
  We will run your server with only one docker command to test the APIs (more on this later)
- **Do not implement an authentication or authorization flow**.

After running the server, all APIs must respond without any authentication.

Understand the Functional Evaluation:

A list of example APIs to help you understand how your database will be populated and how your server functionality will be evaluated through API calls is given here.

For this task, you must submit -

- Full server codebase implementing all required APIs.

Limit usage of AI/LLM generated contents.

AI detection scores exceeding a certain threshold may be penalized.

AI/LLM usage might be overlooked for repetitive or common code components.

Task: Containerize Your Works

Your next task is to make it easily operational for any future system developer who will be responsible for maintenance of the system.

You are required to -

- Dockerize the server
    - Write a minimal dockerfile to containerize your server
- Unify server and database under one single file -
    - Write a single docker compose file to start your server and database.

IMPORTANT: Execution Guidelines

We will test functionality of your submission by running only two commands:

- git clone https-link-to-your-repository.git
- docker compose up --build -d

If that does not work, we will evaluate your submission only based on your submitted codes and designs.

IMPORTANT: Dockerized Database Requirements

- The database must **initialize automatically** with your designed schema when the containers start.
- It will only contain empty tables upon running
- There will be no seed or sample data
- We will populate the database through API calls for testing

{ Hint: This can be achieved by setting one or more SQL files to run at the database container's entrypoint }

For this task, you must submit -

- Dockerfile for your server
- Docker compose file
- SQL file for initial database setup

Call out to the Participants

Each task is eligible for abundant partial marks.

Any submission in the preliminary round, complete or incomplete, will be evaluated sincerely for eligibility to advance to the final round.

Final Submission

Here is a final note as a submission guideline for the preliminary round -

- Create a new github repository.
- **The repository must be created after the problem has been released.**
- **You are not allowed to push any commits to this repository after the deadline. Exceptions will be heavily penalized.**
- In the final round, a list of github users will be provided whom you need to add to your repository as collaborators with editor access for evaluation.

- Setup your project folder in the following manner to submit your works.

root project folder

```
|--- database
|       |--- Init.sql (sql file to initiate database with docker compose)
|--- docs
|       |--- database
|               |--- schemas.md
|--- server
|       |--- (backend codebase)
|       |--- Dockerfile
|--- docker-compose.yaml
```

Best of luck for the preliminary round.